CSS

Cascading Style Sheets

CLASE 8 - HCI 2025

http://www.flickr.com/photos/simonpow/252312738/

Introducción

https://flic.kr/p/2if8cj

Introducción a CSS

- Permite controla el diseño de las páginas Web para que sean más atractivas
- Permite crear reglas que especifican como debe mostrarse el contenido de un elemento
- Por ejemplo, se puede especificar que el fondo de la página debe mostrarse de color crema, los párrafos deben mostrarse de color gris utilizando la tipografía Arial o que todos los encabezados de primer nivel deben mostrarse en color azul, en itálica y con la tipografía Times
- Luego de comprender la forma en la que se escribe una regla CSS, aprender CSS mayoritariamente implica aprender las diferentes propiedades que se encuentran disponibles

Aprendiendo CSS

The Cottage Garden

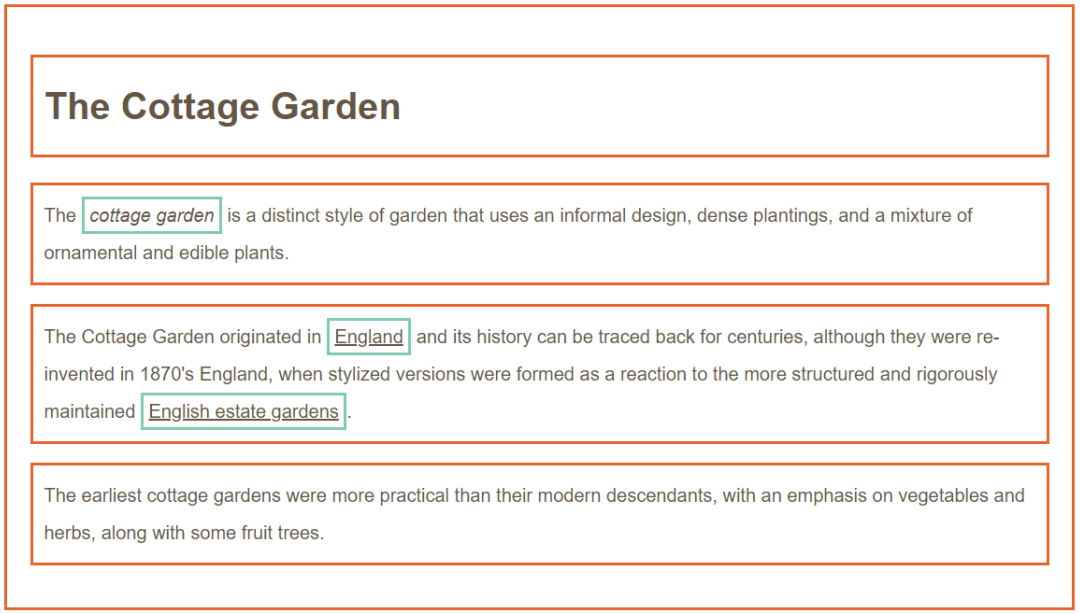
The *cottage garden* is a distinct style of garden that uses an informal design, dense plantings, and a mixture of ornamental and edible plants.

The Cottage Garden originated in England and its history can be traced back for centuries, although they were re-invented in 1870's England, when stylized versions were formed as a reaction to the more structured and rigorously maintained English estate gardens.

The earliest cottage gardens were more practical than their modern descendants, with an emphasis on vegetables and herbs, along with some fruit trees.

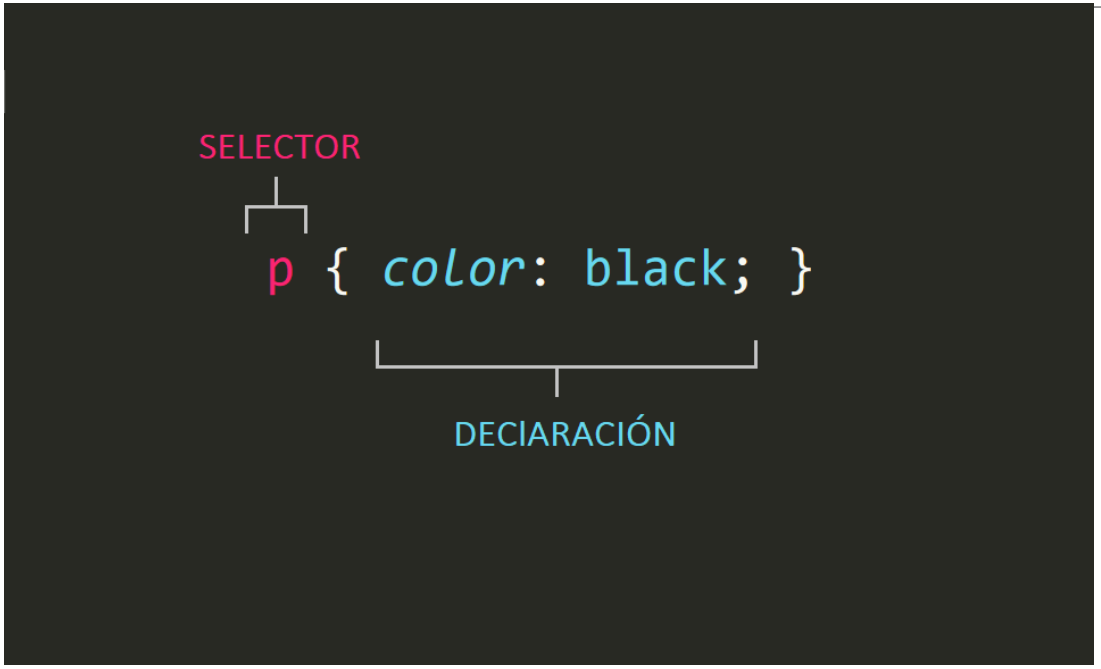
- La clave para comprender como funciona CSS es imaginar que existe una caja invisible alrededor de cada elemento HTML
- Los elementos en bloque comienzan en una nueva línea (por ejemplo: los elementos `<h1>` a `<h6>`, `<p>` y `<div>`)
- Los elementos en línea fluyen dentro del texto y no comienzan en una nueva línea (por ejemplo: elementos `<img>`, `<em>` y `<span>`)

Aprendiendo CSS



- En este ejemplo, los elementos de bloque se muestran con bordes rojos y los elementos en línea con bordes verdes
- La primera caja corresponde al elemento `<body>`, dentro de la misma las otras cajas corresponden a los elementos `<h1>`, `<h2>`, `<p>`, `<em>` y `<a>`
- Usando CSS es posible crear reglas para agregar bordes alrededor de las cajas, especificar su ancho y alto o agregar un color de fondo. También es posible controlar el texto que se encuentran dentro de la caja, alterando su color, tamaño o tipografía

CSS relaciona reglas de estilo con elementos HTML



- Estas reglas definen como el contenido de un elemento debe mostrarse
- Una regla de CSS contiene dos partes: el **selector** y la **declaración**
- El selector indica los elementos a los cuales la regla aplica
- La declaración indica como el elemento referenciado por el selector debe mostrarse

Esta regla indica que el texto de todos los elementos `<p>` debe visualizarse en color negro

Las propiedades CSS definen como se muestran los elementos

```
h1, h2, h3 {
    font-family: Arial;
    color: black;
}
```

PROPIEDAD VALOR

- Las declaraciones de CSS se ubican dentro de corchetes y están formadas de dos partes: una **propiedad** y un **valor**, separados por dos puntos
- Las propiedades indican los aspectos del elemento que se desean cambiar (por ejemplo: color, tipografía, ancho, alto o borde)
- Los valores especifican la configuración que se desea utilizar para las propiedades
- Es posible especificar varias propiedades en una misma declaración, cada una separada de la otra por un punto y coma

Esta regla indica que todos los elementos **<h1>**, **<h2>** y **<h3>** deben mostrarse con la tipografía Arial y en color negro

Usando CSS externo

`<link>`

- El elemento `<link>` puede ser utilizado en un documento HTML para indicarle al navegador donde se encuentra el archivo CSS que se desea utilizar para agregar estilo a la página
- Se trata de un elemento vacío (no necesita etiqueta de cierre) y se ubica dentro del elemento `<head>`
- Debe contener los siguientes atributos:
  - `href`: especifica la ruta al archivo CSS (el cual se ubica generalmente en una carpeta con el nombre css o styles)
  - `rel`: especifica la relación que existe entre la página HTML y el archivo que se referencia. Su valor debe ser **stylesheet** cuando se referencie un archivo CSS
  - `type`: especifica el tipo de archivo al que se hace referencia. Su valor debe ser **text/css**. A partir de HTML 5 ya no es necesario especificarlo
- Una página HTML puede tener más de un archivo CSS (por ejemplo: uno para aspectos comunes como los colores y tipografía y otra para aspectos particulares como la disposición)

Usando CSS interno

`<style>`

- También se pueden incluir reglas CSS dentro de la página HTML ubicando las mismas dentro del elemento `<style>` el cual se ubica dentro del elemento `<head>`
- El atributo `type` especifica el tipo de archivo al que se hace referencia. Su valor debe ser **text/css**. A partir de HTML 5 ya no es necesario especificarlo
- Cuando se construye un sitio Web de más de una página, siempre se debe utilizar CSS externo. Hacerlo permite:
  - Que las páginas hagan uso de las mismas reglas de estilo (en lugar de repetirlas en cada página)
  - Cambiar el estilo a lo largo de todas las páginas alternado un único archivo (en lugar de cada página individualmente)
  - Separar el contenido de como este se visualiza
- En HTML 4 es posible utilizar el atributo `style` en la mayoría de los elementos que forman parte del cuerpo de la página. Las reglas CSS que se encuentren como valor de este atributo solo aplican a dicho elemento. El uso de este atributo no es recomendado

Selectores

Selector	Significado	Ejemplo
Universal	Aplica a todos los elementos del documento	`* {}` Aplica a todos los elementos
Tipo	Aplica a los elementos de un determinado nombre	`h1, h2, h3 {}` Aplica a los elementos `<h1>`, `<h2>` y `<h3>`
Clase	Aplica a los elementos cuyo atributo `class` tengan un valor que coincida con el especificado luego del símbolo punto	`.note {}` Aplica a los elementos cuyo atributo `class` tenga el valor note

Selectores

Selector	Significado	Ejemplo
ID	Aplica al elemento cuyo atributo id coincida con el especificado luego del símbolo numeral	`#introduction {}` Aplica al elemento cuyo atributo `id` tenga el valor introduction
Hijo	Aplica a los elementos que son hijos directos de otro elemento	`li>a {}` Aplica a los elementos `<a>` que sean hijos directos de los elementos `<li>`
Descendiente (espacio)	Aplica a los elementos que se encuentran anidados (no necesitan ser hijos directos) dentro de otro elemento	`p a {}` Aplica a los elementos `<a>` que estén anidados (en cualquier nivel) dentro de los elementos `<p>`

Selectores

Selector	Significado	Ejemplo
Hermano Adyacente	Aplica al primer elemento que es hermano adyacente de otro elemento	`h1+p {}` Aplica al elemento `<p>` que es hermano (en el mismo nivel jerárquico) de los elementos `<h1>`
Hermano General	Aplica a los elementos que son hermanos de otro elemento (no necesitan ser adyacentes)	`h1~p {}` Aplica a los elementos `<p>` que sean hermanos de los elementos `<h1>`

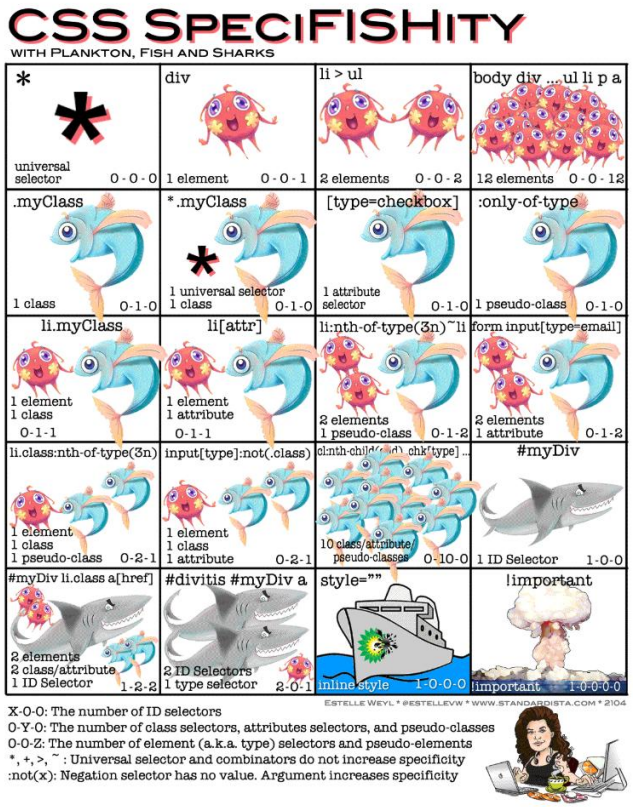
Cascada

- Si existen dos o más reglas que aplican al mismo elemento es importante comprender de que forma toman precedencia
  - **Última regla**: si los dos selectores son idénticos, el último en aparecer toma precedencia
  - **Especificidad**: si un selector es más específico que otros, entonces toma precedencia por sobre el resto
  - **Importante**: agregar `!important` luego de cualquier valor de propiedad permite indicar que esa regla debe ser considerada más importante que las otras que puedan aplicar al elemento
- Comprender como las reglas de CSS funcionan, permite escribir hojas de estilo más simples porque es posible escribir reglas genéricas que aplican a la mayoría de los elementos y luego anular las propiedades de aquellos elementos que necesitan mostrarse de forma diferente

Herencia

- Si se especifica la propiedad `font-family` o `color` al elemento `<body>`, estas se aplicarán a todos los elementos hijos
- Sucede porque los valores de estas propiedades se heredan a sus elementos hijos
- De esta forma se evita tener que aplicar la misma propiedad a múltiples elementos (y como resultado se logra una hoja de estilo más simple)
- Otras propiedades como `background-color` o `border`, no son heredadas por sus elementos hijos
- Algunas propiedades permiten forzar la herencia del valor de su elemento padre, utilizando el valor `inherit` de la propiedad
- Para más detalles consultar https://www.w3.org/Style/CSS/all-properties.en.html

Especificidad



https://specifishity.com/

Diferentes versiones de CSS

- De la misma manera que han existido varias versiones de HTML, también existen diferentes versiones de CSS
- CSS1 fue publicado en 1996 y CSS2 dos años más tarde
- Se ha avanzado mucho sobre CSS3 y varios navegadores ya lo han comenzado a implementar
- Los navegadores no implementan todas las características de CSS al mismo tiempo y algunos navegadores antiguos no soportan todas las propiedades

Particularidades de los navegadores Web

- Antes de publicar un sitio Web, es importante probarlo en más de un navegador debido a que pueden existir pequeñas diferencias en como los navegadores muestran las páginas
- También es una buena idea probarlo en diferentes sistemas operativos y en versiones antiguas de los navegadores más frecuentes al igual que en versiones recientes
- Al ver el sitio en más de un navegador, es posible que algunos elementos de la página no se vean como uno esperaría
- Cuando una propiedad CSS no se muestra como debiera, se dice que el navegador tiene una particularidad (*browser quirk* o *CSS bug*)

Introducción
Resumen

- CSS trata cada elemento HTML como si estuviera dentro de una caja y usa reglas para indicar como debe visualizarse
- Las reglas están formadas por selectores (especifican los elementos al que la regla aplica) y declaraciones (indican de qué forma deben visualizarse los elementos)
- Diferentes tipos de selectores permiten aplicar reglas a diferentes elementos
- Las declaraciones están formadas por dos partes: las propiedades de los elementos que se desean cambiar y los valores que deben tomar esas propiedades (por ejemplo: la propiedad font-family permite cambiar la tipografía y el valor arial especifica que Arial es la tipografía principal)
- Las reglas de CSS habitualmente se ubican en un documento separado

Introducción
Ejemplo

Introducción
Ejemplos

- Cajas invisibles alrededor de los elementos
- Cajas invisibles alrededor de los elementos (con bordes)
- Usando CSS interno
- Usando CSS externo (CSS)
- Selectores CSS
- Cascada de reglas CSS
- Herencia
- Resumen (CSS)

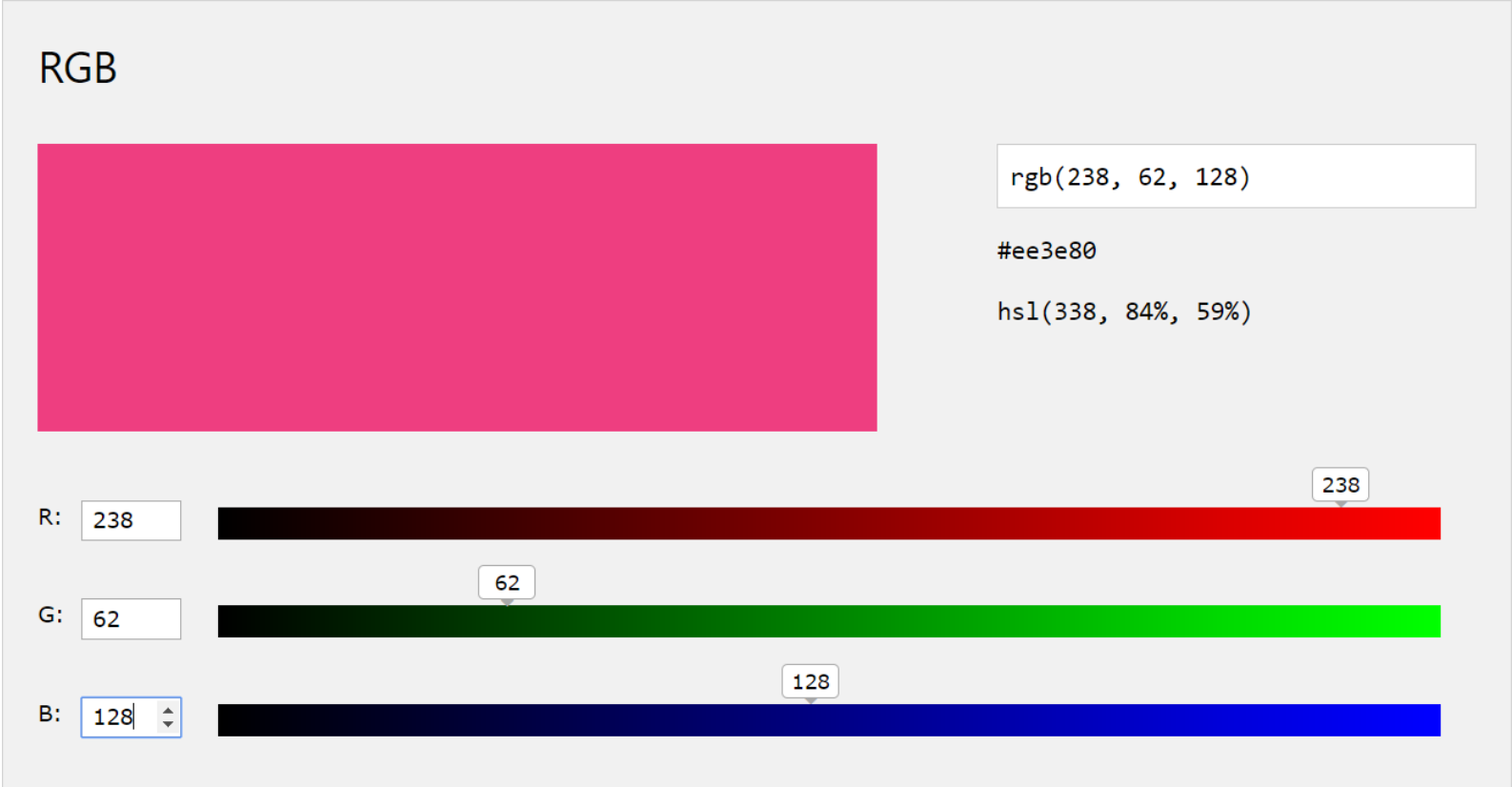
Colores

https://flic.kr/p/638iYQ

Color de primer plano y de fondo `color` y `background-color`

- La propiedad `color` permite especificar el color del texto presente dentro del elemento
- La propiedad `background-color` permite especificar el color de fondo de las cajas que representan a los elementos de HTML
- Si no se especifica el color de fondo, se muestra transparente
- Por defecto, los navegadores muestran una ventana con color de fondo blanco, y les ofrecen a los usuarios la posibilidad de cambiarlo
- En CSS un color se puede especificar de tres formas distintas:
  - **Valores RGB**: representan el color a partir de las cantidades de rojo, verde y azul necesarias para formarlo (por ejemplo: `rgb(100,100,90)`)
  - **Código Hexadecimal**: son códigos de seis posiciones que representan las cantidades de rojo, verde y azul en el color, precedidas por el símbolo numeral (por ejemplo: `#ee3e80`)
  - **Nombres**: existen 147 nombres de colores predefinidos que son reconocidos por los navegadores (por ejemplo: `DarkCyan`)

Color RGB



Contraste

- A la hora de elegir los colores de primer plano y de fondo es importante asegurarse que exista el suficiente contraste como para que el texto pueda ser legible
- La falta de contraste no solo afecta a las personas con discapacidades visuales, sino también a aquellas que tienen pantallas de baja resolución o el sol incidiendo sobre las mismas
- Cuando se traten de textos extensos, reducir levemente el contraste mejora la legibilidad del mismo

Opacidad

`opacity` y `rgba`

- CSS3 incorporó la propiedad `opacity` la cual permite especificar la opacidad de un elemento y de sus hijos
- El valor de la misma es un número entre 0.0 y 1.0 (por lo tanto, un valor de 0.5 representa el 50% de la opacidad)
- CSS3 permite especificar el color mediante un valor RGB, y además permite indicar la opacidad en un cuarto valor (`rgba`)
- Este último valor es conocido como el valor **alpha** y es un número entre 0.0 y 1.0
- El valor `rgba` especificado solo tendrá efecto en el elemento donde sea aplicado
- Debido a que algunos navegadores no reconocen los colores RGBA, es común anteponer a la regla que haga uso del valor `rgba` otra regla que indique un color sólido

HSL & HSLA

`hsl` y `hsla`

- CSS3 incorpora el valor `hsl` como forma alternativa para especificar los colores
- Los valores que puede tomar esta propiedad son:
  - **Matiz**: representado como un ángulo (0 a 360)
  - **Saturación**: expresada como un porcentaje (0% a 100%)
  - **Luminosidad**: también representada como un porcentaje (0% a 100%)
- El valor `hsla` permite especificar en un cuarto valor la transparencia (al igual que el valor `rgba`)
- Debido a que algunos navegadores antiguos no reconocen los valores `hsl` y `hsla`, es una buena práctica agregar una regla extra que especifiquen el color en formato de código hexadecimal, RGB o nombre. Esta regla debe aparecer antes de las regalas que utilizan HSL o HSLA. Esto proporciona una contingencia porque al aplicar las dos reglas al mismo elemento, la última de las dos será la que tome mayor prioridad. Y en el caso de que no se encuentre soportada, se tomará la primera

Color HSL

Colores
Resumen

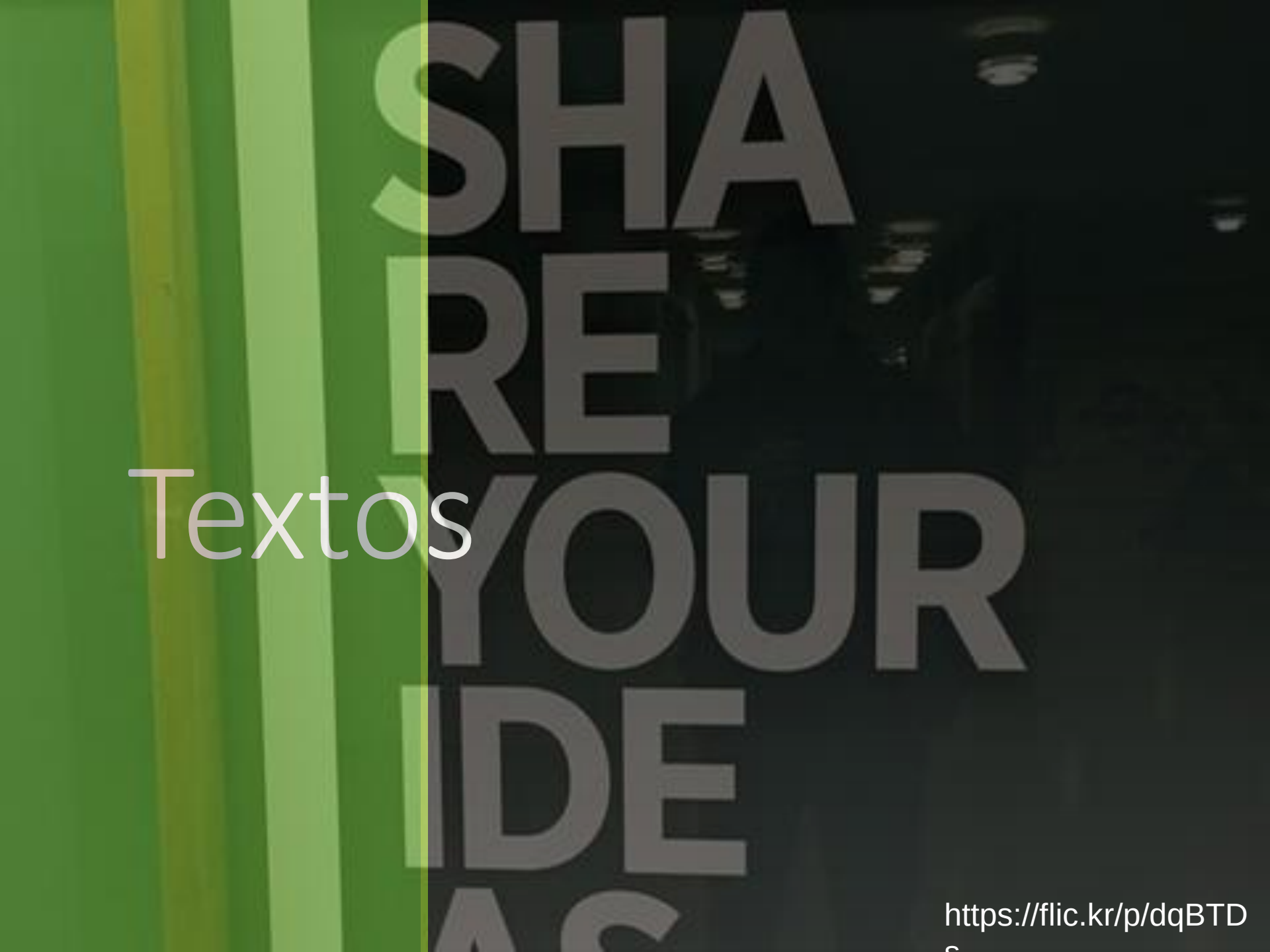
- El color no solo le da vida al sitio, sino que también transmite el estado de ánimo y genera reacciones
- Existen 3 formas de especificar color en CSS: valores RGB, códigos hexadecimales y nombres
- Los selectores de color son de gran ayuda a la hora de encontrar el color deseado
- Es importante asegurarse que exista el suficiente contraste entre el texto y color de fondo (caso contrario será difícil leer el contenido)
- CSS3 introduce un valor extra para los colores RGB, el cual permite indicar la opacidad (conocido como RGBA)
- CSS3 también permite especificar colores como valores HSL, con un valor opcional de opacidad (conocido como HSLA)

Colores
Ejemplo

Colores
Ejemplos

- Color de primer plano
- Color de segundo plano
- Opacidad
- HSLA
- Resumen

Textos



https://flic.kr/p/dqBTD

Tipografías

- Las tipografías ***Serif*** (Georgia, Times, Times New Roman) tienen ciertos detalles en los trazos principales de las letras conocidos como *serifs*
- Las tipografías *Serif* son utilizadas en fragmentos extensos de texto impreso porque se considera que son más fáciles de leer
- Las tipografías ***Sans-serif*** (Arial, Verdana, Helvetica) tienen líneas rectas y por lo tanto un diseño más limpio
- Las pantallas tienen menos resolución que las impresiones, por lo tanto, cuando el texto sea pequeño las tipografías *Sans-serif* serán más fáciles de leer
- Cada letra en una tipografía ***Monospace*** (Courrier, Courrier New) tiene el mismo ancho.
- Las tipografías *Monospace* son comúnmente utilizadas para representar código, haciendo que el mismo sea más fácil de seguir.

Indicando la tipografía
font-family

- La propiedad `font-family` permite especificar la tipografía usada por el texto que se encuentra dentro de los elementos a los que aplique la regla
- Las personas que visiten el sitio Web necesitan que la tipografía especificada se encuentre instalada en sus computadoras para que la misma se muestre
- Es posible especificar una lista de tipografías separadas por comas, de tal manera que, si el usuario no tiene instalada la primera opción de tipografía, se pueda probar con las otras alternativas de la lista
- También es frecuente terminar la lista con un nombre genérico de tipografía de acuerdo al tipo (por ejemplo: serif o sans-serif)
- Si el nombre de la tipografía se encuentra formado por dos o más palabras, es necesario escribir el mismo entre comillas dobles

Tamaño de la tipografía
font-size

- La propiedad `font-size` permite especificar el tamaño de la tipografía
- Existen distintas formas de hacerlo:
  - **Píxeles**: su uso es frecuente debido a que permiten controlar en forma precisa el tamaño que ocupará el texto. La cantidad de píxeles deseada debe estar acompañada de las letras px. La cantidad de píxeles es relativa a la resolución de la pantalla, por lo tanto, se debe tener en cuenta que cuanto más alta la resolución más pequeño se mostrará el texto
  - **Porcentajes**: el tamaño por defecto de la tipografía en los navegadores es de 16px. Por lo tanto, un tamaño del 75% es el equivalente a 12px y 200% a 32px
  - **EMS**: un em es el equivalente al ancho de la letra m
- Debido a que los usuarios pueden cambiar en los navegadores Web el tamaño por defecto de la tipografía, de usar las dos últimas formas existe la posibilidad de que las mismas se muestren más grandes o pequeñas que lo diseñado

Tipografías no instaladas
`@font-face`

- La regla `@font-face` permite utilizar una tipografía, incluso si la misma no estuviera instalada en la computadora de las personas que visiten el sitio Web, especificando una ruta de acceso a la misma para que pueda ser descargada e instalada
- La propiedad `font-family` permite especificar el nombre de la tipografía. Este valor es el que luego se utiliza como valor de la propiedad `font-family` en el resto de la hoja de estilo
- La propiedad `src` permite especificar la ruta de acceso a la tipografía. Debido a que no todos los navegadores soportan los mismos formatos de tipografía es posible que se deban especificar diferentes formatos de la misma
- La propiedad `format` permite especificar el formato de la tipografía (por ejemplo: svg, ttf, etc.)
- Varios diseñadores de tipografías no permiten que las mismas sean utilizadas en sitios Web. Por lo tanto, antes de hacer uso de una tipografía verificar la licencia de uso de la misma (debe permitir su uso en forma comercial).

Pseudo-elementos y pseudo-clases

- El **pseudo-elemento** actúa como si existiera un elemento extra en el código HTML
- En el caso de los pseudo-elementos `:first-letter` y `:first-line`, es como si hubiera un elemento cuyo contenido son la primera letra y la primera línea del texto a los cuales se les puede aplicar su propio estilo
- La **pseudo-clase** actúa como un valor adicional para el atributo `class`
- La pseudo-clase `:visited`, permite aplicar un estilo diferente para los hipervínculos que no fueron visitados, mientras que `:hover` permite hacerlo cuando el usuario se posiciona con el cursor sobre el elemento
- Tanto los pseudo-elementos como las pseudo-clases se especifican al final de un selector (por ejemplo: `p.into:first-letter { font-size: 200%; }` o `a:visited { color: black; }`)

Selectores de atributo

Selector	Significado	Ejemplo
Existencia	`[]` Aplica a todos los elementos que contengan el atributo (sin impotar su valor)	`p[class]` Aplica a todos los elementos `<p>` que contengan el atributo `class`
Equivalencia	`[=]` Aplica a todos los elementos que contengan el atributo con el valor especificado	`p[class="dog"]` Aplica a todos los elementos `<p>` que contengan el atributo `class` y su valor sea dog

Selectores de atributo

Selector	Significado	Ejemplo
Espacio	`[~=]` Aplica a todos los elementos que contengan el atributo y el valor especificado forme parte de una lista de valores separadas por espacios	`p[class~="dog"]` Aplica a todos los elementos `<p>` que contengan el atributo `class` y su valor sea una lista de palabras separadas por espacios y una de ellas sea dog
Prefijo	`[^=]` Aplica a todos los elementos que contengan el atributo y el valor del mismo comience con el valor especificado	`p[class^="d"]` Aplica a todos los elementos `<p>` que contengan el atributo `class` y su valor comience con “d”

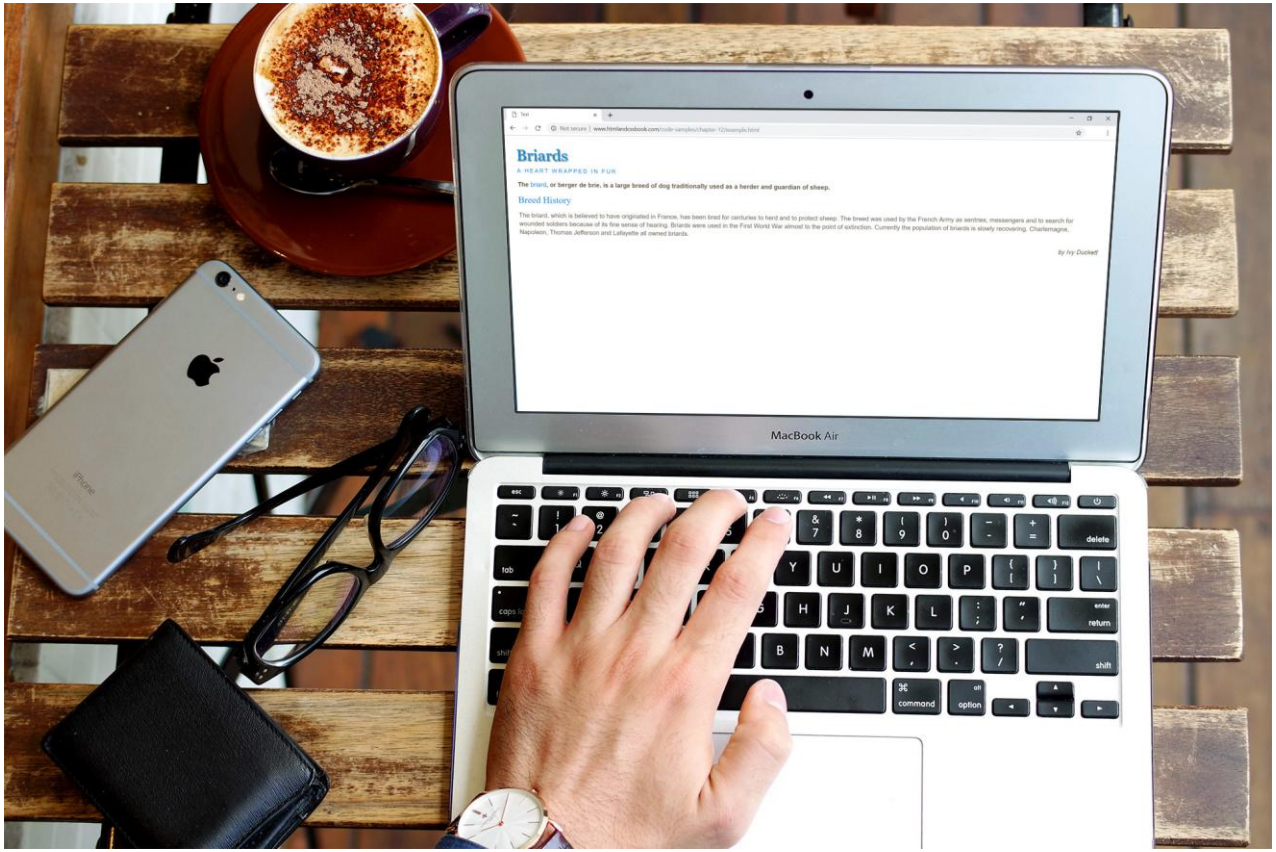
Selectores de atributo

Selector	Significado	Ejemplo
Subcadena	`[*=]` Aplica a todos los elementos que contengan el atributo y el valor del mismo contenga con el valor especificado	`p[class*="do"]` Aplica a todos los elementos `<p>` que contengan el atributo `class` y su valor contenga “do”
Sufijo	`[$=]` Aplica a todos los elementos que contengan el atributo y el valor del mismo termine con el valor especificado	`p[class$="g"]` Aplica a todos los elementos `<p>` que contengan el atributo `class` y su valor termine con “g”

Textos
Resumen

- Existen propiedades para controlar la elección de la tipografía, tamaño, peso, estilo y espaciamiento
- Existe una limitada cantidad de tipografías que se pueden asumir que la mayoría de las personas tendrá instaladas
- Si se desea utilizar una gama más amplia de tipografías existen varias opciones, pero es necesario contar con la licencia adecuada para poder utilizarlas
- Se puede controlar el espaciamiento entre las líneas de texto, las letras o las palabras. El texto también puede ser alineado a izquierda, derecha, centro o justificado. También puede ser indentado (sangría)
- Se puede utilizar pseudo clases para cambiar el estilo de un elemento cuando el usuario se posicione sobre el mismo, cuando haga clic sobre el mismo o haya visitado un hipervínculo

Textos
Ejemplo



Textos
Ejemplos

- Familia de tipografías
- Tamaño de la tipografía
- Usando tipografías no instaladas
- Relevancia de la tipografía
- Estilo de la tipografía
- Transformación de textos
- Decoración de textos
- Altura de la línea
- Espaciamiento de letras y palabras

Textos
Ejemplos

- Alineamiento de texto
- Alineamiento vertical
- Sangría
- Sombras
- Primera línea y letra
- Foco y activación
- Resumen

Cajas

https://flic.kr/p/VdhAnG

Dimensiones de las cajas

`height` y `width`

- Por defecto las cajas son dimensionadas para poder almacenar su contenido
- Para modificar este comportamiento es possible utilizar las propiedades `height` y `width`
- Las formas más frecuentes para especificar las dimensiones de una caja son:
  - **Píxeles**: son los más difundidos porque le permiten a los diseñadores controlar el tamaño con precisión
  - **Porcentajes**: el tamaño de la caja es relativo al tamaño de la ventana del navegador o en caso de que se encuentre contenida dentro de otra caja, al porcentaje del tamaño de la caja que la contiene
  - **EMS**: el tamaño de la caja es relativo al tamaño del texto contenido dentro de la misma
- Las formas relativas se han popularizado últimamente, para permitir crear diseños que sean flexibles a lo largo de dispositivos con diferentes tamaños de pantalla

Limitando las dimensiones

`min-width`, `max-width`, `min-height`, `max-height` y `overflow`

- Algunas páginas se expanden y contraen para cubrir el tamaño de la pantalla del usuario
- En estos diseños, la propiedad `min-width` especifica el tamaño mínimo que una caja puede tener cuando se contrae la ventana del navegador Web y la propiedad `max-width` especifica el tamaño máximo que la caja puede tener cuando se expande la ventana del navegador Web
- De una forma similar se puede controlar el alto de la caja utilizando las propiedades `min-height` y `max-height`
- Cuando el tamaño de una caja no es suficiente para almacenar su contenido, este se expande por fuera de la misma. Para controlar en estas situaciones la forma en la que debe mostrarse el mismo se puede utilizar la propiedad `overflow`
- Estas propiedades son muy útiles para garantizar que el contenido de las páginas sea legible
- Algunas de estas propiedades no funcionan correctamente en versiones antiguas de los navegadores Web

Borde, margen y relleno

- Cada caja tiene tres propiedades disponibles que pueden ser ajustadas para controlar su apariencia
- Cada caja tiene un **borde** (incluso cuando no sea visible). El borde separa los límites de una caja de otra
- Los **márgenes** se encuentran por fuera de los límites del borde. Pueden ser ajustados para crear un espaciamiento entre los bordes de dos cajas adyacentes
- El **relleno** es el espacio entre el borde de una caja y su contenido. Agregar relleno puede incrementar la legibilidad del contenido
- Si se especifican las dimensiones de una caja, el borde, margen y relleno serán agregados al ancho y alto de la misma

Ancho del borde
`border-width`

- La propiedad `border-width` es usada para controlar el ancho del borde
- Puede tomar valores en pixels o tomar alguno de los siguientes valores: `thin`, `medium` o `thick` (no se pueden usar valores relativos en esta propiedad)
- Es posible controlar el ancho de cada uno de los cuatro bordes de la caja utilizando las propiedades `border-top-width`, `border-right-width`, `border-bottom-width` y `border-left-width`
- También es posible especificar diferentes anchos para los cuatro bordes en una sola propiedad de la siguiente manera: `border-width: 2px 1px 1px y 2px;` (los valores deben ingresarse en el sentido horario (arriba, derecha, abajo e izquierda)

Estilo y color del borde

`border-style` y `border-color`

- Se puede controlar el estilo de un borde usando la propiedad `border-style`
- Puede tomar alguno de los siguientes valores: `solid`, `dotted`, `dashed`, `double`, `groove`, `ridge`, `inset`, `outset`, `hidden` / `none`
- Es posible controlar el estilo de cada uno de los cuatro lados de la caja utilizando las propiedades `border-top-style`, `border-right-style`, `border-bottom-style` y `border-left-style`
- Se puede controlar el color del borde usando valores RGB, valores hexadecimales o nombres
- Es posible controlar el color de cada uno de los cuatro lados de la caja utilizando las propiedades `border-top-color`, `border-right-color`, `border-bottom-color` y `border-left-color`
- La propiedad `border` permite especificar el ancho, estilo y color del borde en una única propiedad (los valores deben ingresarse en orden)

Relleno
`padding`

- La propiedad `padding` permite especificar cuanto espacio debe mostrarse entre el contenido del elemento y su borde
- El valor de esta propiedad normalmente se especifica en píxeles (aunque también puede especificarse en porcentaje y ems)
- Si se usa un valor de tipo porcentaje, el relleno se calcula como el porcentaje respecto de la ventana del navegador (o la caja que la contiene en caso de estar contenida dentro de otra caja)
- Tener en cuenta que el relleno forma parte del ancho y alto total de la caja
- Es posible controlar el relleno de cada uno de los cuatro lados de la caja utilizando las propiedades `padding-top`, `padding-right`, `padding-bottom` y `padding-left`
- También es posible especificar diferentes rellenos para los cuatro bordes en una sola propiedad `padding`
- El valor de la propiedad `padding` no es heredado por los elementos hijos

Margen
`margin`

- La propiedad `margin` permite especificar el espacio que debe mostrarse entre las cajas
- El valor de esta propiedad normalmente se especifica en píxeles (aunque también puede especificarse en porcentaje y ems)
- Si una caja se solapa con otra, los márgenes se muestran colapsados. Esto significa que el más largo de los dos será utilizado y el más corto ignorado
- Tener en cuenta que el margen forma parte del ancho y alto total de la caja
- Es posible controlar el relleno de cada uno de los cuatro lados de la caja utilizando las propiedades `margin-top`, `margin-right`, `margin-bottom` y `margin-left`
- También es posible especificar diferentes márgenes para los cuatro bordes en una sola propiedad `margin`
- El valor de la propiedad `margin` no es heredado por los elementos hijos

Centrando contenido

- Si se desea centrar una caja respecto de la página o de otra caja que la contenga, se puede asignar el valor `auto` a las componentes `left-margin` y `right-margin`
- También es necesario asignarle un ancho a la caja
- De esta forma el navegador establecerá un espaciamiento equivalente a cada lado de la caja permitiendo que la misma quede centrada respecto de la página o de la caja que contenga a la misma.

Tipo y visibilidad
`display` y `visibility`

- La propiedad `display` permite transformar un elemento de tipo en línea al tipo en bloque y viceversa y además puede ser utilizada para remover un elemento de la página
- Puede tomar alguno de los siguientes valores:
  - `inline`: transforma un elemento al tipo en línea
  - `block`: transforma un elemento al tipo en bloque
  - `inline-block`: transforma un elemento del tipo en bloque al tipo en línea conservando algunas de sus características (por ejemplo: ancho y alto)
  - `none`: elimina un elemento de la página. Se lo trata como si nunca hubiera existido, a pesar de que se encuentra presente el código de la página.
- La propiedad `visibility` permite ocultar el contenido de una caja dejando el espacio que esta ocuparía.
- Puede tomar alguno de los siguientes valores: `hidden` o `visible`

Cajas
Resumen

- CSS trata a cada elemento HTML como si tuviera su propia caja
- Se pude utilizar CSS para controlar las dimensiones de las cajas
- También se pueden controlar los bordes, los márgenes y el espaciamiento de cada caja con CSS
- Es posible ocultar elementos usando las propiedades `display` y `visibility`
- Cajas a nivel de bloque pueden transformarse en cajas en línea y viceversa
- La legibilidad puede mejorarse controlando el ancho de las cajas que contienen textos
- CSS3 incorpora la posibilidad de crear bordes redondeados y a partir de imágenes

Cajas
Ejemplo

Cajas
Ejemplos

- Dimensiones de las cajas
- Limitando el ancho de las cajas
- Limitando el alto de las cajas
- Controlando el exceso de contenido
- Ancho del borde
- Estilo del borde
- Color del borde
- Borde

Cajas
Ejemplos

- Relleno
- Margen
- Cambiando el tipo de caja
- Ocultando cajas
- Aplicando imágenes a los bordes
- Aplicando sombras a los bordes
- Bordes redondeados
- Resumen

Listas, tablas y formularios

https://flic.kr/p/ozbtX

Listas, tablas y formularios
Resumen

- Complementando las propiedades de CSS vistas anteriormente las cuales aplican al contenido de todos los elementos, existen otras tantas que son específicas para controlar la apariencia de listas, tablas y formularios
- Se les pueden aplicar diferentes apariencias a las viñetas de las listas usando las propiedades `list-style-type` y `list-style-image`
- Si bien las celdas de las tablas pueden tener diferentes bordes y espaciamiento a lo largo de los diferentes navegadores, existen propiedades que permiten controlarlos y hacerlos más consistentes
- Los formularios son más fáciles de usar si sus controles se encuentran verticalmente alineados
- Los formularios se benefician del estilo ya que los hace parecer más interactivos

Listas, tablas y formularios
Ejemplo

Listas, tablas y formularios
Ejemplos

- Estilo de las viñetas
- Imágenes como viñetas
- Posicionando las viñetas
- Propiedades de las tablas
- Celdas vacías
- Espaciamiento de las celdas
- Aplicando estilo a los cuadros de texto
- Aplicando estilo a los botones

Listas, tablas y formularios
Ejemplos

- Aplicando estilo a las leyendas
- Alineando los controles del formulario
- Cursores
- Resumen

Disposición

https://flic.kr/p/5phTQq

Bloques de construcción

- CSS trata a cada elemento HTML como si estuviera contenido dentro una caja invisible
- Estas pueden ser de dos tipos: **en bloque** o **en línea**
- Las cajas en bloque comienzan en una nueva línea (por ejemplo: los elementos `<h1>` a `<h6>`, `<p>` y `<div>`) y representan los bloques de construcción principales de cualquier diseño
- Las cajas en línea fluyen dentro del texto y no comienzan en una nueva línea (por ejemplo: elementos `<img>`, `<em>` y `<span>`)
- Se puede especificar cuanto espacio debe ocupar cada caja mediante las propiedades `width` y `height`
- Para separar las cajas se pueden utilizar bordes, márgenes, relleno y colores de fondo

Contenedor de elementos

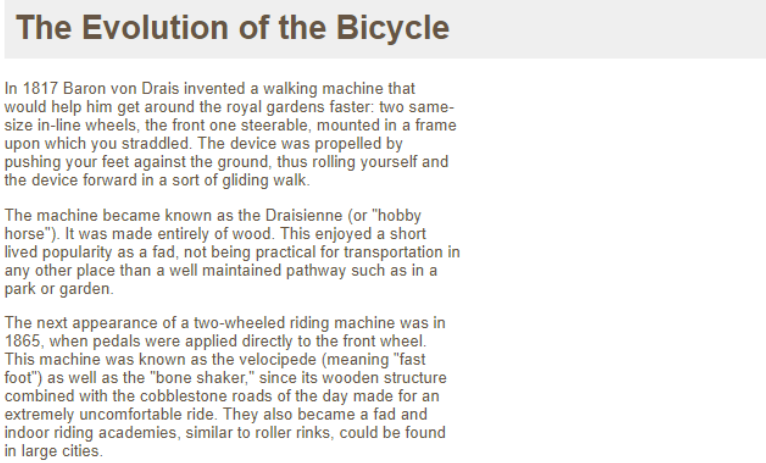
- Si un elemento en bloque se encuentra ubicado dentro de otro elemento en bloque el elemento padre se denomina **contenedor** o elemento **padre**
- Es una práctica común agrupar un conjunto de elementos dentro de un `<div>` u otro elemento en bloque
- Por ejemplo: se pueden agrupar todos lo elementos que componen el encabezado de un sitio Web (logo, barra de navegación, etc.)
- Este elemento en bloque que contiene al grupo de elementos es referido como el elemento contenedor

Controlando la posición de los elementos

- CSS contempla los **esquemas de posicionamiento** flujo normal, posicionamiento relativo y posicionamiento absoluto los cuales permiten controlar la disposición de los elementos que componen la página
- El esquema de posicionamiento se puede especificar utilizando la propiedad `position`
- También es posible que los elementos floten utilizando la propiedad `float`
- Para determinar el lugar donde la caja debe posicionarse, también se deberán utilizar las propiedades de desplazamiento, las cuales le indican al navegador a que distancia de la parte superior o inferior e izquierda o derecha debe posicionarse la caja
- Cuando se desplaza un elemento de su flujo normal, las cajas se pueden superponer. La propiedad `z-index` permite especificar cuales cajas debe ser mostradas en primer plano

Flujo normal

- Cada elemento en bloque se muestra en una nueva línea, causando que cada elemento se muestre en la página más abajo que el anterior
- Si no se especifica el ancho de un elemento en bloque este ocupará todo el ancho de su elemento contenedor
- Si se especificara el ancho de las cajas de tal forma que el espacio permita que dos elementos se muestren uno al lado del otro, los mismos se seguirán mostrando uno debajo del otro
- Este es el comportamiento por defecto que tiene el navegador a menos que se le indique que haga otra cosa. Por lo tanto no es necesario asignar el valor `static` a la propiedad `position`



The Evolution of the Bicycle

In 1817 Baron von Drais invented a walking machine that would help him get around the royal gardens faster: two same-size in-line wheels, the front one steerable, mounted in a frame upon which you straddled. The device was propelled by pushing your feet against the ground, thus rolling yourself and the device forward in a sort of gliding walk.

The machine became known as the Draisienne (or "hobby horse"). It was made entirely of wood. This enjoyed a short lived popularity as a fad, not being practical for transportation in any other place than a well maintained pathway such as in a park or garden.

The next appearance of a two-wheeled riding machine was in 1865, when pedals were applied directly to the front wheel. This machine was known as the velocipede (meaning "fast foot") as well as the "bone shaker," since its wooden structure combined with the cobblestone roads of the day made for an extremely uncomfortable ride. They also became a fad and indoor riding academies, similar to roller rinks, could be found in large cities.

Posicionamiento relativo

- Mueve el elemento respecto de la posición que este hubiera ocupado en el flujo normal, desplazándolo hacia arriba, abajo, izquierda o derecha de acuerdo a lo solicitado
- El desplazamiento del elemento no afecta a los elementos que lo rodean, los mismos mantienen la posición que ocupan en el flujo normal
- Para especificar un posicionamiento relativo es necesario asignar el valor `relative` a la propiedad `position`
- Las propiedades de desplazamiento (`top` o `bottom` y `left` o `right`) permiten indicar cuan lejos se debe desplazar el elemento de su posición en el flujo normal

The Evolution of the Bicycle

In 1817 Baron von Drais invented a walking machine that would help him get around the royal gardens faster: two same-size in-line wheels, the front one steerable, mounted in a frame upon which you straddled. The device was propelled by pushing your feet against the ground, thus rolling yourself and the device forward in a sort of gliding walk.

The machine became known as the Draisienne (or "hobby horse"). It was made entirely of wood. This enjoyed a short lived popularity as a fad, not being practical for transportation in any other place than a well maintained pathway such as in a park or garden.

The next appearance of a two-wheeled riding machine was in 1865, when pedals were applied directly to the front wheel. This machine was known as the velocipede (meaning "fast foot") as well as the "bone shaker," since it's wooden structure combined with the cobblestone roads of the day made for an extremely uncomfortable ride. They also became a fad and indoor riding academies, similar to roller rinks, could be found in large cities.

Posicionamiento absoluto

- Mueve el elemento respecto de la posición del elemento contenedor, desplazándolo hacia arriba, abajo, izquierda o derecha de acuerdo a lo solicitado
- El elemento es removido del flujo normal por lo tanto a diferencia de lo que sucede en el esquema de posicionamiento relativo los otros elementos se comportan como si este no estuviera presente
- Estos elementos se mueven cuando el usuario se desplaza verticalmente en la página
- Para especificar un posicionamiento relativo es necesario asignar el valor `absolute` a la propiedad `position`
- Las propiedades de desplazamiento (`top` o `bottom` y `left` o `right`) permiten indicar cuan lejos se debe desplazar el elemento respecto de la posición del elemento contenedor

In 1817 Baron von Drais invented a walking machine that would help him get around the royal gardens faster: two same-size in-line wheels, the front one steerable, mounted in a frame upon which you straddled. The device was propelled by pushing your feet against the ground, thus rolling yourself and the device forward in a sort of gliding walk.

The machine became known as the Draisienne (or "hobby horse"). It was made entirely of wood. This enjoyed a short lived popularity as a fad, not being practical for transportation in any other place than a well maintained pathway such as in a park or garden.

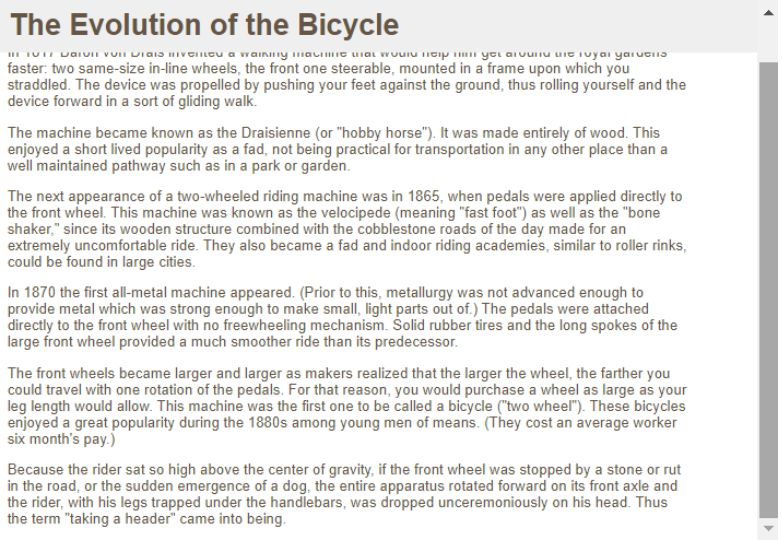
The next appearance of a two-wheeled riding machine was in 1865, when pedals were applied directly to the front wheel. This machine was known as the velocipede (meaning "fast foot") as well as the "bone shaker," since its wooden structure combined with the cobblestone roads of the day made for an extremely uncomfortable ride. They also became a fad and indoor riding academies, similar to roller rinks, could be found in large cities.

In 1870 the first all-metal machine appeared. (Prior to this, metallurgy was not advanced enough to provide metal which was strong enough to make small, light parts out of.) The pedals were attached directly to the front wheel with no freewheeling mechanism. Solid rubber tires and the long

The Evolution of the Bicycle

Posicionamiento fijo

- Es una variante del posicionamiento absoluto el cual se posiciona al elemento respecto de la ventana del navegador, en lugar de la posición del elemento contenedor
- El elemento es removido del flujo normal por lo tanto a diferencia de lo que sucede en el esquema de posicionamiento relativo los otros elementos se comportan como si este no estuviera presente
- Estos elementos a diferencia de lo que sucede con el esquema de posicionamiento absoluto no se mueven cuando el usuario se desplaza verticalmente en la página
- Para especificar un posicionamiento relativo es necesario asignar el valor `fixed` a la propiedad `position`
- Las propiedades de desplazamiento (`top` o `bottom` y `left` o `right`) permiten indicar cuan lejos se debe desplazar el elemento respecto de la posición del elemento contenedor

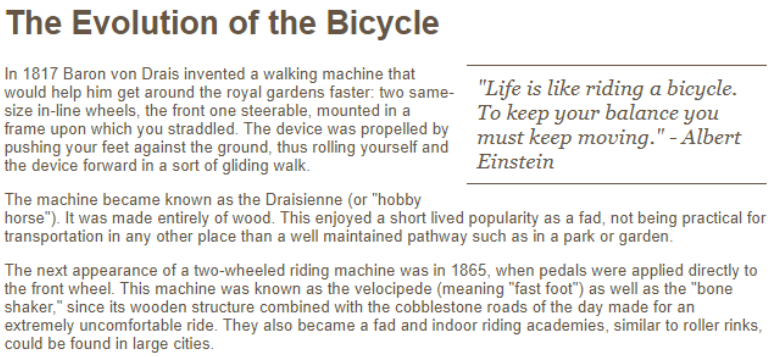


Superposición de elementos `z-index`

- Cuando se usa un esquema de posicionamiento relativo, fijo o absoluto, las cajas pueden superponerse
- Si las cajas se superponen, estas son los elementos que aparecen luego en el código HTML se posicionan por encima de aquellas que aparecen antes en la página
- Para controlar la forma en que se deben superponer los elementos se debe utilizar la propiedad `z-index` (representa un eje z invisible que sale de pantalla)
- Esta propiedad toma un valor numérico y cuanto mayor sea el mismo, más cerca del frente se encontrará el elemento. Por ejemplo: un elemento con valor 10 de z-index se visualizará por encima de un elemento con valor 5 de z-index

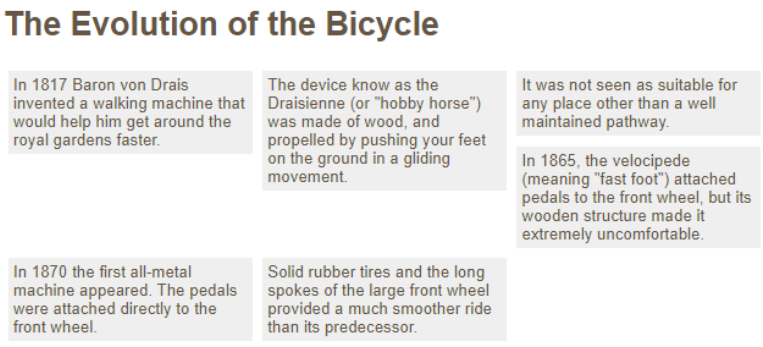
Elementos flotantes
`float`

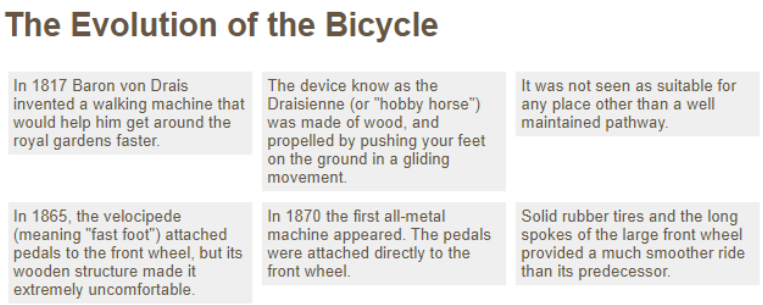
- La propiedad `float` permite que un elemento en el flujo normal pueda ser posicionado sobre el margen izquierdo o derecho del elemento contenedor
- Cualquier otro contenido que se encuentre en el elemento contenedor va a fluir alrededor del elemento flotante
- Cuando se utiliza la propiedad `float` es necesario especificar un valor en la propiedad `width` para indicar el ancho que debe tener el elemento flotante (si no se lo hace el elemento posiblemente ocupe todo el ancho del elemento contenedor)



Elementos flotantes
`clear`

- Cuando los elementos flotan, la altura de las cajas puede afectar la posición de los subsiguientes elementos
- La propiedad `clear` permite especificar que ningún elemento (dentro del mismo contenedor) debe tocar el borde izquierdo, derecho o ambos de otra caja





Diseño de ancho fijo

- No cambian a medida que el usuario incrementa o decrementa el tamaño de la ventana de su navegador
- La unidad de medida más utilizadas en estos diseños es el píxel
- **Ventajas**:
  - Los valores en píxeles son precisos a la hora de controlar el tamaño y posición de los elementos
  - El diseñador tiene un gran control respecto a la apariencia y posicionamiento de los elementos de la página
  - Se pueden controlar las longitudes de las líneas de los textos
  - El tamaño de las imágenes seguirá siendo el mismo
- **Desventajas**:
  - Se pueden presentar zonas con falta de contenido en los bordes de la página
  - Si la resolución es muy grande, la página se verá más pequeña y por lo tanto los textos serán más difíciles de leer
  - Si el usuario incrementa el tamaño de la tipografía es posible que los textos no quepan dentro de los elementos
  - La página posiblemente requiera más espacio vertical que un diseño responsivo para mostrar el mismo contenido

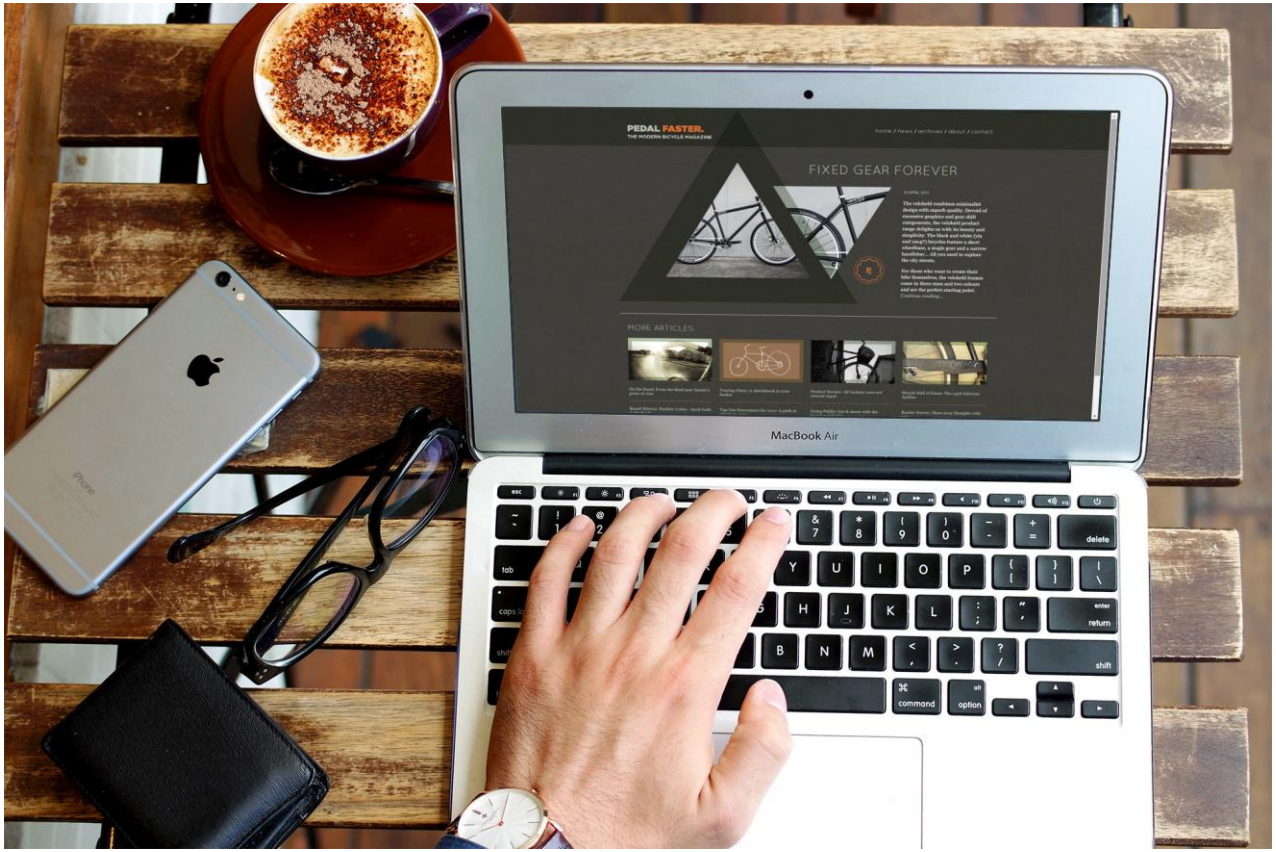
Diseño responsivo

- Se expande y contrae a medida que el usuario incrementa o decrementa el tamaño de la ventana de su navegador
- La unidad de medida más utilizadas en estos diseños es el porcentaje
- **Ventajas**:
  - En pantallas grandes, la página se adapta ocupando todo el tamaño de la ventana del navegador
  - En pantallas pequeña, la página se contrae para ocupar la misma minimizando que el usuario tenga que desplazarse verticalmente
  - El diseño es tolerante a que el usuario incremente el tamaño de la tipografía
- **Desventajas**:
  - Al no controlar el tamaño de algunas secciones de la página, el diseño puede verse muy distinto de como debería verse, con espaciamientos alrededor de algunos elementos o elementos aplastados
  - Si el usuario tiene una pantalla ancha, las líneas de texto pueden ser muy extensas dificultando su lectura
  - Si el usuario tiene una pantalla angosta, las palabras pueden aplastarse y terminar con líneas de pocas palabras

Disposición
Resumen

- Los elementos `<div>` se utilizan como contenedores de otros elementos para agrupar secciones de la página
- Los navegadores muestran los elementos de la página en flujo normal a menos que se especifique un posicionamiento relativo, absoluto o fijo
- La propiedad `float` mueve el contenido del elemento hacia la izquierda o derecha de la página y puede ser utilizada para crear disposiciones de múltiples columnas
- Las páginas pueden tener un diseño de ancho fijo o responsivo
- Los diseñadores tratan de que las páginas tengan entre 960 y 1000 píxeles de ancho e indican de que trata el sitio en los primeros 600 píxeles (para demostrar su relevancia sin que sea necesario desplazarse verticalmente)
- Los sistemas de grillas permiten crear diseños flexibles y profesionales.
- Los Frameworks de CSS proporcionan reglas para las tareas más comunes
- Es posible incluir múltiples archivos CSS en una misma página

Disposición
Ejemplo



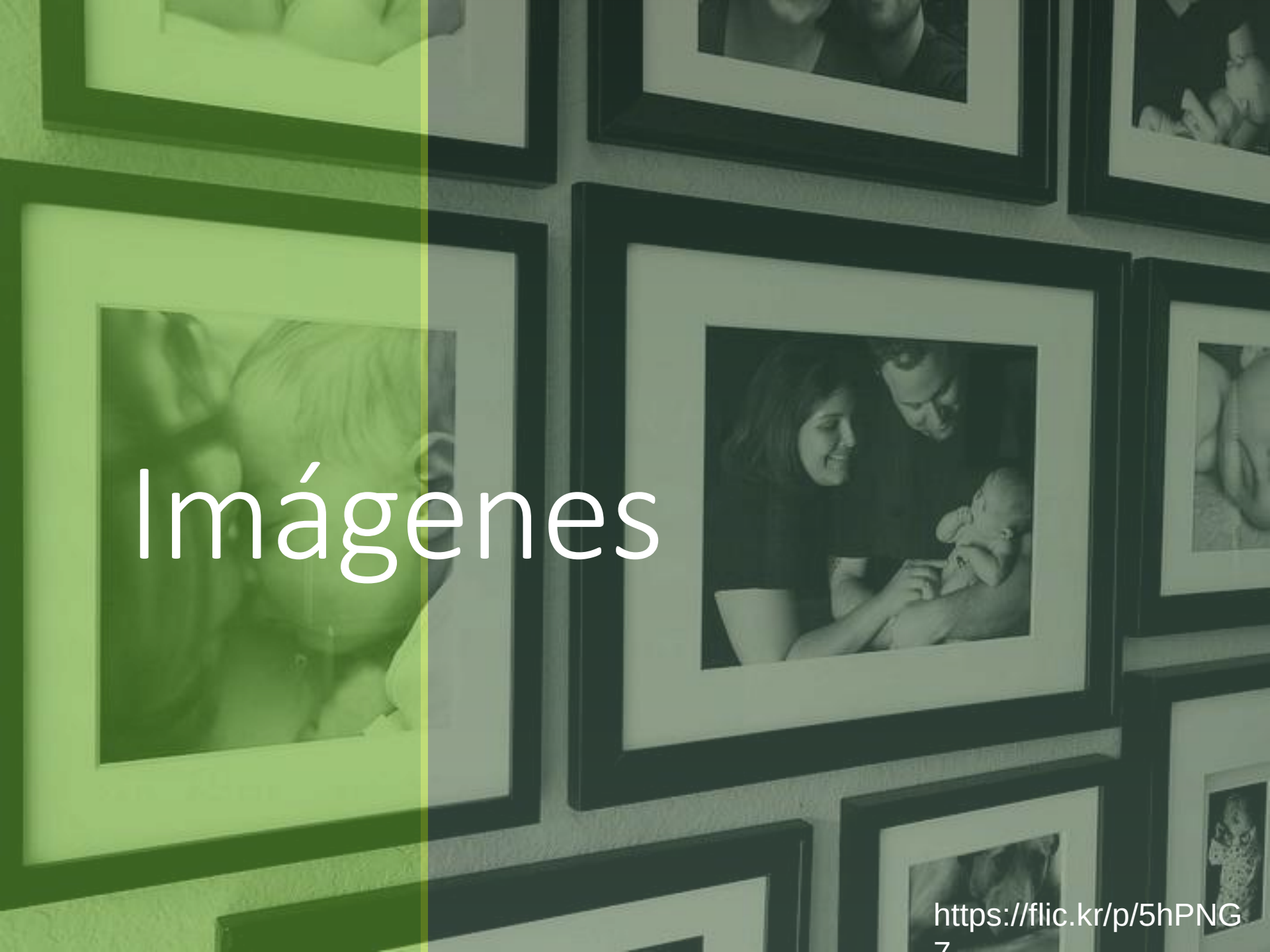
Disposición
Ejemplos

- Flujo normal
- Posicionamiento relativo
- Posicionamiento absoluto
- Posicionamiento fijo
- Superposición de elementos
- Superposición de elementos (z-index)
- Diseño de ancho fijo
- Diseño fluido (responsivo)

Disposición
Ejemplos

- Múltiples hojas de estilo (import)
- Múltiples hojas de estilo (link)
- Resumen

Imágenes



https://flic.kr/p/5hPNG

Dimensiones
`height` y `width`

- El tamaño de una imagen se puede modificar usando las propiedades `height` y `width` (de la misma forma que con cualquier otra caja)
- Especificar las dimensiones de las imágenes permite que las páginas carguen fluidamente
- Debido a que el HTML y CSS se carga antes que las imágenes, especificarle al navegador cuanto espacio debe generar para representar cada imagen permite que el resto de la página no se vea afectada por la carga de las mismas
- Cuando se utilicen tamaños de imágenes en forma consistente a lo largo de todo el sitio es preferible utilizar CSS para controlar sus dimensiones en lugar del HTML (por ejemplo: generando clases small, medium o large)

Alineación

- Para alinear horizontalmente una imagen se puede utilizar la propiedad `float`
- Para evitar que el texto de los párrafos esté en contacto con los bordes de la imagen se debe agregar un valor a la propiedad `margin` a la imagen
- Por defecto las imágenes son elementos en línea. Por lo tanto, para poder centrarlas es necesario convertirlas en elementos en bloque utilizando la propiedad `display`
- Luego es posible centrarlas asignando el valor `center` a la propiedad `text-align` o `auto` a los valores izquierdo y derecho de la propiedad `margin`
- Cuando se utilicen tamaños de imágenes en forma consistente a lo largo de todo el sitio es preferible utilizar CSS para controlar sus dimensiones en lugar del HTML (por ejemplo: generando clases small, medium o large)

Fondo de los elementos

`background-image`, `background-repeat` y `background-attachment`

- La propiedad `background-image` permite que un elemento pueda tener una imagen de fondo
- Por defecto la imagen se repetirá hasta cubrir completamente las dimensiones de la caja
- La ubicación de la imagen se especifica mediante la palabra `url` seguida de la ruta que permite acceder a la misma entre paréntesis y comillas
- La propiedad `background-repeat` puede tomar alguno de los siguientes valores: `repeat` (equivalente a repeat-x y repeat-y), `repeat-x`, `repeat-y` o `no-repea`
- La propiedad `background-attachment` permite especificar si la imagen de fondo debe quedar fija o desplazarse a medida que el usuario se desplaza verticalmente por la página
- Puede tomar dos valore: `fixed` o `scroll`

Fondo de los elementos
`background-position` y `background`

- Cuando una imagen de fondo no se repite se puede utilizar la propiedad `background-position` para especificar la ubicación de la misma
- Esta propiedad generalmente tiene dos valores, el primero representa la posición horizontal y el segundo la vertical (por ejemplo: left top o right bottom). Cuando se especifica un solo valor se toma por defecto `center` para el segundo valor
- También es posible especificar estos valores en píxeles o porcentaje
- La propiedad `backround` permite especificar el color, imagen, forma de repetición, desplazamiento y posición

Imágenes
Resumen

- Se puede especificar las dimensiones de las imágenes utilizando CSS. Esto es muy útil cuando se utilizan imágenes del mismo tamaño en diferentes páginas del sitio Web
- Las imágenes pueden alinearse tanto horizontalmente como verticalmente con CSS.
- Los navegadores muestran los elementos de la página en flujo normal a menos que se especifique un posicionamiento relativo, absoluto o fijo
- Se puede utilizar una imagen de fondo detrás de la caja creada por cualquier elemento de la página
- Las imágenes de fondo pueden mostrarse una sola vez o repetirse a lo largo y ancho del fondo de la caja
- Para reducir el número de imágenes que el navegador debe descargar se pueden combinar las mismas en una única imagen (sprites)

Imágenes
Ejemplo

Imágenes
Ejemplos

- Dimensiones
- Alineación izquierda y derecha
- Alineación centro
- Fondo página
- Fondo elemento
- Fondo repetición
- Fondo desplazamiento
- Fondo posición

Imágenes
Ejemplos

- Fondo
- Resumen